Reading the Mountain Before It Slides: An AWS Avalanche-Safety Pipeline

Ebin Joseph
Student ID: X25142224
MSc in Cloud Computing
Fog and Edge Computing (H9FECC)
National College of Ireland
x25142224@student.ncirl.ie

Abstract*—A seismic spike that precedes a slab release lasts on the order of minutes, far shorter than the interval between a ski patrol's scheduled slope checks, so any monitoring approach built around periodic human inspection is already too slow by design. This paper builds and deploys a fog-to-cloud pipeline that instead watches two simulated slopes continuously: five sensor types per slope, including a seismic-vibration channel standing in for the acoustic precursor signal literature associates with avalanche release, feed a plain Node.js HTTP fog gateway with no web framework, whose outbound SQS client is an ES6 Proxy that only constructs the real AWS client on first use and rebuilds it whenever the gateway is reconfigured. Aggregated windows are batched into Amazon SQS rather than sent one message at a time, drained by an AWS Lambda function into DynamoDB, and served back out through a second, independently deployed Lambda behind an Amazon API Gateway REST API, dispatching on a hand-written array of path templates rather than a regular expression, a trie, or a third-party router. The complete system was pushed to a real AWS Academy account, where a DynamoDB pagination undercount, a missing SQS batching capability, a credential-handling risk in deploy tooling, a credential-gating bug that only manifested once the Lambda functions were actually invoked in production, and a missing CORS header that left the live dashboard rendering nothing despite a healthy API underneath, were all found and fixed against the running deployment rather than assumed correct from a passing local test suite.

***Keywords*—fog computing, edge computing, Internet of Things, serverless architecture, AWS Lambda, cloud deployment, avalanche safety**

I. Introduction

Avalanche danger does not build up gradually in a way a scheduled walk-through would reliably catch. A rising wind loading a lee slope, a warming snowpack losing its bond to the layer beneath it, or a burst of seismic activity consistent with a slab starting to fail can each develop and pass within the gap between two patrol rounds. Instrumenting the slope itself, rather than relying on a person to walk it, is what closes that gap: sensors that never stop sampling can flag a threshold crossing the moment it happens, not whenever a human next arrives to look.

That instrumentation only helps if the data it produces reaches a decision point fast enough to matter, which is where fog computing enters the design. Bonomi et al. define fog computing as an extension of the cloud that pushes computation, storage, and networking services toward the edge of the network, close to where devices and users actually are [1]; a slope-side sensor array is exactly this kind of edge deployment, geographically dispersed and only intermittently able to reach a distant data center. Cloud computing itself is characterised by the National Institute of Standards and Technology around five traits: on-demand self-service, broad network access, resource pooling, rapid elasticity, and measured service [2]. None of those five traits say anything about what should happen physically at the slope, which is precisely the gap this project's fog node fills: it buffers raw readings, closes a window on a fixed timer, evaluates four alert rules against that window's aggregate, and only then hands a small, already-summarised message to the cloud side of the system, so the elastic backend in Section II never has to absorb a continuous raw sensor stream at all.

Three components had to be built and then proven working, not just described, for this project to count as finished. A sensor-and-fog tier needed to buffer, window, and evaluate readings for two slopes at whatever pace each of five sensor types was configured to run, not a rate hard-coded into the ingestion path. A backend needed to turn each closed window into a stored, queryable record using managed AWS services rather than a server this project has to keep patched itself. And none of that would count until it was actually running against a real AWS account and checked there directly, since a design that only works inside a local Docker Compose file has not yet proven anything about the cloud it claims to target.

Section II lays out the architecture at each layer along with the alternatives that were weighed and set aside, and closes on the security posture the deployed system settles into. Section III moves to implementation -- the software stack, the automated test suite, continuous integration, and the real AWS deployment itself, defects included. Section IV then evaluates the system using evidence pulled directly from that live deployment, and Section V assesses the project critically and sketches what is left for future work.

II. System Architecture and Design

A. Sensor and Fog Layer

Two simulated slopes, slope-a and slope-b, each carry five sensor types: snowpack depth (snowpack_depth_cm), snow temperature (snow_temp_c), wind speed (wind_speed_kmh), seismic vibration (seismic_vibration_mg, a stand-in for the acoustic precursor signal avalanche-monitoring literature associates with slab failure [3]), and lift chair count (lift_chair_count, an operational rather than a hazard signal). Each sensor process runs two independent, AbortController-coordinated setTimeout loops rather than a single combined timer: one samples a bounded random walk into a local outbox at SAMPLE_INTERVAL, the other drains that whole outbox in one POST at DISPATCH_INTERVAL, restoring the batch to the front of the outbox if the request fails so nothing sampled while a dispatch was in flight is lost or reordered.

The fog node buffers every incoming reading in a bare object literal keyed on sensor_type::site_id, not a Map, an array, or a ring buffer -- addReading() appends straight into station.groups[key], and on a fixed WINDOW_SECONDS timer, snapshotAndClear() walks that object with Object.keys(), packages each non-empty key into a group, and

resets station.groups to a fresh {} so readings that land immediately afterward start a new window rather than leaking into the one just closed. Each closed group is reduced to five figures -- count, minimum, maximum, average, and the last-arrived reading -- and checked against Table I's four alert rules before it ever leaves the fog node.

Sensor Type	Rule	Alert Key
seismic_vibration_mg	avg > 25	avalanche_risk_detected
wind_speed_kmh	avg > 80	lift_wind_halt
snow_temp_c	avg > 2	snowpack_instability_risk
snowpack_depth_cm	avg < 30	insufficient_snow_coverage

TABLE I. ALERT THRESHOLDS (EVALUATED ON THE WINDOW AGGREGATE)

Every rule in Table I is a Rule class instance, not a row in a generic tuple array, a case in a switch statement, or a value in a dispatch object: each instance owns one field, one comparison operator, limit, alert key, and sensor type, and carries a check() method that tests one summary directly against them. evaluateAlerts() is exactly RULES.filter(r => r.check(summary)).map(r => r.key) -- no lookup table sits between a closed window and the rule that judges it. lift_chair_count deliberately has no Rule instance at all, so it is reported to the dashboard for operational visibility but can never itself trigger an alert.

The fog gateway's HTTP surface -- /health, /thresholds, and POST /ingest -- is dispatched through a switch(true) statement over a composed `${method} ${pathname}` string, deliberately avoiding Express, an if/else chain, a tuple table, or a routing trie. Node's event-driven, non-blocking I/O model is what makes a plain http server a reasonable choice here at all: Tilkov and Vinoski describe how Node.js trades a thread-per-connection model for a single event loop plus asynchronous I/O, favouring workloads with many concurrent, short-lived connections over CPU-bound work [4], which matches this gateway's job of accepting frequent small POSTs from ten sensor processes rather than performing heavy computation.

B. Scalable Backend Layer

The backend owns no server this project has to provision or patch. Sealed windows are published to ska-slope-agg, an Amazon SQS queue, and an AWS Lambda function -- ska-processor -- is invoked automatically by an event-source mapping whenever messages arrive, writing each aggregate straight into DynamoDB. Both the queue and the function scale to whatever volume the two slopes actually produce, without a fixed pool of worker processes sitting idle between flush cycles.

sensor_type is the table's partition key, but two slopes closing the same sensor type in the same flush window would otherwise collide on an identical sort key. transform.js resolves this with sort_key = window_end#site_id: folding site_id into the sort key rather than using a separate global secondary index keeps every slope's reading for a given window in its own row under the same partition, retrievable with a single Query rather than a Scan plus a filter -- the same partition-key-plus-derived-attribute pattern DeCandia et al. describe as central to how Amazon's own internal Dynamo store achieves single-digit-millisecond reads at scale [5].

Cold starts are the standard critique levelled at Lambda-based designs -- one Shahrad et al.'s large-scale characterisation of a real production serverless workload found significant enough to shape how that workload gets scheduled and provisioned [6] -- and this pipeline is exposed to them in two different places with two different consequences. ska-processor is queue-triggered, so a slow cold start only delays how quickly a buffered message is drained, never how quickly a browser renders anything, since nothing is waiting on that invocation directly. ska-dashboard-api sits on the read path a browser actually waits on, but its own container tends to stay warm on its own: the dashboard polls it every 2.5 seconds while a page is open, and that polling cadence is frequent enough in practice to keep a cold start from being the common case.

C. A Template-Matcher Dispatch for the Dashboard Lambda

Locally, the dashboard's read endpoints are served by server.js, a plain Node http server dispatching on the same switch(true)-over-composed-key idiom the fog gateway uses. That object never exists inside a real Lambda invocation -- there is no listening socket for it to be attached to, only a single incoming event object -- so a second, independently deployed Lambda, ska-dashboard-api, answers the real deployment's Amazon API Gateway REST API instead.

Its dispatch shape is a plain array of {method, template, handler} entries, matched by walking the request path and each candidate template together, rather than a regular-expression scan, a character trie, a flat dictionary keyed on the exact method and path, or a switch expression over a concatenated string. template is an ordinary string such as /api/slopes/:slopeId with a leading colon marking a path parameter. matchTemplate() splits both the template and the actual request path on '/' and walks the two segment lists together: a colon-prefixed template segment always matches and captures whatever segment sits in the same position, and a literal segment must match exactly or the whole route is rejected. findRoute() simply walks ROUTES in order and returns the first match, so adding a new endpoint means appending one array entry, not touching a regular expression or a lookup table maintained separately from the routes themselves.

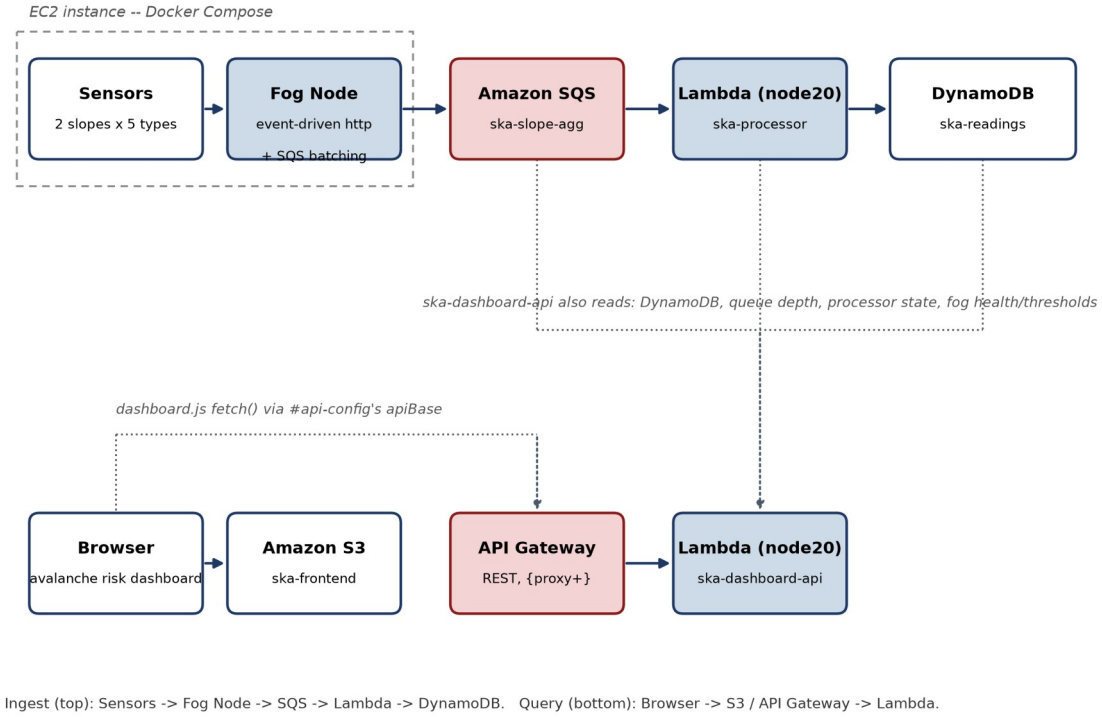

Fig. 1. Deployment topology. Sensors, fog node, SQS, ska-processor, and DynamoDB form the ingest path along the top; browser, S3, API Gateway, and ska-dashboard-api form the query path along the bottom. Only the EC2-hosted sensor and fog tier sits outside a managed AWS service.

D. Deployment Topology

The boundary Fig. 1 draws is the one this deployment actually enforces. The EC2 instance carrying sensors and the fog node is the only piece sitting outside a managed AWS service, and even that instance holds no SSH key pair -- AWS Systems Manager Session Manager is the sole way in, and its security group admits nothing but the fog node's own port 8000. The top row of the figure is the ingest side: sensors post to the fog node, the fog node batches closed windows onto SQS, SQS triggers ska-processor, and ska-processor writes to DynamoDB. The bottom row is the query side: a browser pulls the static dashboard from S3, then makes its own separate call straight to API Gateway, which forwards it to ska-dashboard-api -- the one component in the whole diagram that reads back across both halves, since answering a single dashboard request means pulling from DynamoDB plus the three other sources the figure names.

An unencrypted port on a bare IP address is precisely the traffic shape corporate and campus firewalls are tuned to drop, so serving the dashboard itself from that same EC2 address would have left it unreachable from exactly the network a demo audience is likely to be on. S3 plus API Gateway avoids that problem outright: both already speak HTTPS on 443, and neither one cares whether the EC2 instance is reachable, or even powered on, when a request arrives.

E. Critical Analysis of Alternative Architectures

Several designs were weighed against the ones adopted above and set aside for a specific, stated reason rather than left unconsidered. The fog node's plain object-literal buffer was chosen over a Map keyed the same way: at this project's scale -- two slopes, five sensor types, at most ten distinct keys open at once -- a bare object's Object.keys() walk costs nothing measurable, and the trade-off only starts to favour a Map once the key count grows large enough that property lookup on a plain object begins to show its hash-collision overhead, a threshold this deployment never approaches.

A hand-written route table for the fog gateway's HTTP layer was considered in place of the switch(true) dispatch actually used, and rejected because a table sitting apart from the handler functions themselves is one more place a route can silently drift out of sync as endpoints are added -- the switch statement keeps method, path, and handler in the same three lines, with no separate structure to forget to update.

Set aside for the fog-to-processor link: Apache Kafka, whose main selling point over SQS is an ordered log several independent consumer groups can each replay at their own pace, surviving a restart without losing a partition's read position. Nothing about this pipeline needs that guarantee. ska-processor is the only consumer of these messages, DynamoDB itself already orders each stored item by sort_key, and SQS's event-source mapping wires straight into Lambda with no broker cluster to size, no partitions to plan, and no consumer-group client to configure at all -- a simpler match for one queue with one always-on reader.

Also set aside: collapsing ska-processor and ska-dashboard-api into one Lambda, so ingestion and query serving share a single function. The two have incompatible load shapes. ska-processor's demand tracks whatever backlog happens to be sitting in the SQS queue at a given moment; ska-dashboard-api answers one browser request at a time and has to return quickly regardless of what the queue is doing. A shared reserved-concurrency limit and a shared timeout would tie those two demands together for no benefit -- a burst of queued messages could leave a dashboard request queued behind a busy ingestion invocation, and a slow stretch of dashboard reads could just as easily starve the queue side of capacity.

Deferred rather than rejected: moving the sensor and fog tier off EC2 onto a scheduled-Lambda model, so nothing in the system runs on a continuously provisioned instance. A scheduled invocation fires cold and stateless every time, which is fine for a queue consumer but works against how this fog node's buffer earns its keep -- it only has value because it stays resident and keeps accumulating readings between flush

cycles, and a scheduled function would force that state out into some external store instead. Cloud deployment in this project was scoped to the backend from the start, so this particular rebuild stays future work rather than something attempted here.

F. Security Considerations

TCP 8000 is the fog node's one inbound opening on this EC2 instance, and it is also the only one: the instance holds no key pair, so SSH is not an option, and every operator action instead goes through AWS Systems Manager Session Manager. What that single open port actually exposes is limited to read-only status data -- aggregated sensor summaries and alert keys -- never a raw per-reading value and never any control surface.

Both Lambda functions run under the one LabRole this Learner Lab account provides, since the account has no path for a student to create a new IAM role of their own; that is an account limitation, not a design decision made freely. A properly split policy would leave ska-dashboard-api holding only read access to DynamoDB, to the queue's attributes, and to Lambda metadata, and would trim ska-processor down to sqs:ReceiveMessage, sqs:DeleteMessage, and dynamodb:PutItem -- a noticeably narrower footprint than the shared role grants either function today out of necessity. The dashboard's REST API carries no authentication layer, which was a considered choice: everything it returns is an aggregated, non-personal reading from a simulated sensor, so there is nothing sensitive an auth check would be guarding. That same API answers any origin via a wildcard CORS header, and Chen et al.'s empirical study of CORS deployments in the wild found that a large fraction of real-world misconfigurations occur at exactly this boundary, either reflecting arbitrary origins back with credentials permitted or leaving internal endpoints reachable cross-origin unintentionally [7] -- a caution this deployment sidesteps only because it never pairs a wildcard origin with credentialed requests in the first place, not because the underlying risk is absent.

III. Implementation

A. Software Components and Libraries

Every component runs on Node.js 20 in plain CommonJS, with no TypeScript build step and no web framework anywhere in the stack -- sensors, fog node, and both Lambda handlers all sit on top of Node's built-in http module or a plain exported handler function. The official AWS SDK for JavaScript v3 (@aws-sdk/client-sqs, client-dynamodb, lib-dynamodb, client-lambda) is the only AWS-facing dependency; Chart.js draws the dashboard's window-average trend charts client-side, vendored locally rather than pulled from a content-delivery network, so the deployed page carries no third-party runtime dependency beyond what ships in its own S3 bucket.

Several implementation-choice axes below -- the fog buffering container, the alert-rule representation, the HTTP dispatch style, and the outbound SQS client's own lazy-construction shape -- carry over patterns worked out in project submissions written earlier than this one. None of that material is an earlier submission of this student's own, and readme.txt's REUSE section discloses exactly where each piece came from rather than leaving it unattributed; each axis named there was read directly from its actual source before being reshaped into something that departs from it in a concrete way, not reused unchanged. The domain logic -- slope sensor profiles, the four alert thresholds in Table I, the risk-level derivation, and the entire dashboard frontend -- is original to this project.

The fog gateway's outbound SQS client is an ES6 Proxy wrapping a lazily constructed SQSClient: reading any property other than a handful of control methods (configure, useClient, reset, publish, publishBatch) triggers the Proxy's get trap, which builds and caches a real client on first access if nothing has already configured or injected one. configure() deliberately nulls the cached client back out, so calling it a second time with a new endpoint rebuilds the client on the next access rather than silently reusing a stale one pointed at the wrong region.

B. Testing Strategy

Each module carries its own package.json and Node's built-in node:test runner, no external test framework anywhere: 13 tests for sensors, 47 for the fog layer, 10 for the backend processor, and 51 for the dashboard. intake.test.js covers the fog buffer's group-at-ingest and snapshot-and-reset behaviour directly, and publisher.test.js exercises the Proxy's lazy construction and queue-url memoization. server.test.js and lambdaHandler.test.js each exercise their respective dispatch layers with real request objects rather than unit-testing the underlying route handlers in isolation: server.test.js drives a real Node http.Server bound to an ephemeral port, while lambdaHandler.test.js calls the exported handler with API-Gateway-shaped event objects and injected fake AWS clients, asserting on the returned {statusCode, headers, body} shape directly.

Two tests exist purely to pin down the defects covered in the next subsection. pipelineStatus.test.js feeds countTableItems() four fake Scan pages -- 512, 340, 289, and 156 items -- and checks the total comes back as 1297, the recursive-generator pagination having actually walked every LastEvaluatedKey rather than stopping at the first page's 512. publisher.test.js checks the other direction: a 23-message flush window has to land as SendMessageBatch calls sized ten, ten, and three, never as twenty-three separate SendMessageCommand calls, and a companion test confirms an empty window triggers no calls at all, not even a wasted queue-URL lookup.

C. Continuous Integration and Deployment

GitHub Actions runs each of the four modules' test suites on every push and pull request, but as four independent jobs rather than one shared build -- each module installs its own dependencies and runs node --test on its own, so a dependency bump in one module cannot quietly take down another module's pipeline. A second stage, gated on the first passing, brings the full LocalStack-backed docker-compose.yml stack up, runs infra/verify_pipeline.py against it end to end, and tears everything back down afterward regardless of outcome, leaving nothing running for a later job to collide with.

D. AWS Deployment and Defects Discovered

By the end of this project, nothing about it was still a local simulation: an AWS Academy Learner Lab account in us-east-1 hosts every piece, from the EC2-based sensor and fog containers through the queue, both Lambda functions, DynamoDB, API Gateway, and the S3-hosted frontend. Getting there surfaced five real defects, split roughly in half between what a code read caught beforehand and what only the live system itself gave away afterward -- a distinction worth keeping, since the second group is exactly the kind of failure a passing local test suite cannot surface on its own.

First, pipelineStatus.js's countTableItems() issued a single Scan call with Select: "COUNT". Scan only counts the page it actually reads, roughly 1MB of scanned data, and signals there is more by attaching a LastEvaluatedKey to the response; a caller that never follows that key reports a number that stops short of the table's true size with nothing to flag the shortfall. The fix, scanCountPages(), is a recursive async generator that yield*-delegates into itself for every LastEvaluatedKey page it receives, summed by a for-await consumer in countTableItems() -- there is no while loop, do-while loop, or manually managed array anywhere in the file.

Second, the fog node's flush cycle originally sent one SendMessageCommand per closed group in a loop, correct but wasteful whenever more than one group closes in the same window, the normal case across two slopes and five sensor types. publisher.publishBatch() now collects an entire window's messages into one list and chunks it at SendMessageBatch's ten-entry limit, and the fog node's flush handler calls it once per window instead of looping the single-message call.

Third, backend/processor/deploy_lambda.sh -- tooling written for deploying against LocalStack specifically -- unconditionally exported AWS_ACCESS_KEY_ID=test and AWS_SECRET_ACCESS_KEY=test regardless of which endpoint it was pointed at, which would have silently overwritten real credentials in the environment had it ever been run against a live account by mistake. It was never exercised in the real deployment -- both Lambda functions were created directly through the AWS CLI, bypassing this script entirely -- but the risk is now documented in the script itself with a comment so a future reader does not assume it is safe to run against a real account.

Fourth, and only found once the real Lambdas were actually invoked in production: awsClients.js, handler.js, and publisher.js all conditionally built an explicit AWS credentials object whenever AWS_ACCESS_KEY_ID was present in the environment, which is correct for the LocalStack profile but wrong once deployed, because AWS Lambda always injects that exact variable itself to carry its own execution role's temporary credentials. Both real Lambda functions were therefore always taking that branch and rebuilding an incomplete credential object missing a session token, and every DynamoDB, SQS, and Lambda call failed outright with UnrecognizedClientException: The security token included in the request is invalid -- confirmed directly in CloudWatch logs, and visible in the deployed /api/health response reporting its queue and lambda fields both false. The fix gates the same conditional on AWS_ENDPOINT_URL instead, a variable that is only ever set for the LocalStack profile and never set by Lambda or on EC2; left unset, each AWS SDK client's default provider chain correctly resolves either Lambda's full injected credential triple or EC2's instance-metadata role. Redeploying both functions and rechecking /api/health confirmed the fix: all four fields returned true within seconds, and the roughly 160-message SQS backlog that had built up during the outage drained completely once reprocessing succeeded.

Fifth, lambdaHandler.js's responses carried no Access-Control-Allow-Origin header at all. The S3-hosted frontend's cross-origin fetch() calls were consequently blocked by the browser's same-origin enforcement before dashboard.js's rendering code ever saw a response, and because tick()'s catch block swallows a fetch failure into a bare retry with no logging, the deployed page loaded with zero console errors and zero failed requests visible in the network tab, while every panel stayed empty. The API itself was never at fault: curl against the same endpoint, with no Origin header to trip the browser's enforcement, returned real, live data throughout. The defect was caught only by loading the deployed page in an actual browser and watching every panel stay blank despite that. The fix adds Access-Control-Allow-Origin: "*" to every response lambdaHandler.js returns, consistent with Section II-F's security posture -- this API serves aggregated, non-personal data with no authentication layer by design, so a public origin introduces nothing a determined reader could not already retrieve directly.

None of the five fixes were taken on faith from the unit test suite alone; each got a second check against the running deployment itself. The pagination and batching assertions -- the four-page sum and the ten/ten/three chunk split -- exercise the same code path a real DynamoDB or SQS client would run, so the fake-backed test result and the live behaviour are not two separate claims. The credential-gating and CORS fixes had no equivalent unit-level proxy available, so confirming them meant redeploying the Lambda code itself, polling /api/health afterward, and reloading the deployed page in a browser until every panel filled with live data. Full setup and deployment notes live in readme.txt; the source, defects and fixes included, sits at [GitHub repository URL].

IV. Evaluation

A. Automated Test Coverage

All 121 tests, broken down by module in Table II, pass against the LocalStack-backed development setup, and the handful that touch real client-construction logic were checked a second time against the fixes confirmed on the live AWS deployment covered in Section III-D.

Module	Tests	Focus
sensors	13	bounded random walk, dual-rate sample/dispatch loops
fog	47	object-literal buffer, Proxy publisher, SQS batching, real-socket /ingest
backend/processor	10	sort-key disambiguation, DynamoDB item shape
backend/dashboard	51	routes, pagination fix, real-socket + Lambda dispatch tests

TABLE II. AUTOMATED TEST SUITE BY MODULE

B. Live Deployment Health

Two kinds of evidence back the claim that this deployment actually works, not just that it was configured. The first is what the system says about itself: /api/health, polled after the credential-gating fix, reported gateway, queue, lambda, and pipeline all true, with freshest_age_seconds under a second. The second is independent of that self-report entirely -- direct AWS CLI queries against each resource, none of them routed through the application's code. get-function confirmed State Active for both Lambdas; get-event-source-mapping confirmed State Enabled for the queue driving ska-processor; describe-instances confirmed the EC2 host running, bound to its Elastic IP, with inbound access limited to TCP 8000 and no key pair attached; and describe-table confirmed DynamoDB ACTIVE under on-demand billing.

A plain Scan against DynamoDB -- deliberately not the application's possibly-buggy count logic, the source of one of the five defects -- put the item count at 0 immediately after the credential fix and 569 roughly ninety seconds later, the SQS backlog from the earlier outage draining into stored rows in real time. Later checks kept climbing further as live sensor traffic continued, which is what a genuinely running pipeline looks like rather than a frozen number. The dashboard itself was then opened in a browser rather than only probed with curl: page, stylesheet, vendored chart library, and dashboard.js all returned 200 with no console errors, both slopes' risk gauges and reading panels refreshed with changing values on each 2.5-second poll, and the alert banner correctly lit up when a live wind_speed_kmh reading crossed the lift_wind_halt threshold.

C. Cost Considerations

Almost everything in this stack bills per request rather than per provisioned capacity, which keeps it inside AWS's free-tier allowances at the volume this project actually generates: DynamoDB on-demand pricing, both Lambda functions well under the perpetual one-million-request free tier, API Gateway and SQS charged the same per-request way with nothing accruing between calls. The one exception is the EC2 instance, billed continuously because it hosts the fog node and sensor containers as long-running processes rather than short invocations. Since DynamoDB, Lambda, and API Gateway are what actually keeps the dashboard and its API answering, turning that EC2 instance off between demo sessions does not take either down -- it only flips the health check's gateway flag to false and pauses new readings until the instance is running again.

D. Limitations

None of the following four gaps should be left for a reader to discover on their own. The sensor and fog tier still runs on one EC2 instance, so it alone carries a single point of failure while everything downstream of it sits on redundant AWS infrastructure; Section II-E already explains why a fully serverless version of this tier was deferred, but a deferred fix leaves the availability gap exactly as real. The whole system also ran inside an AWS Academy Learner Lab account built for bounded class sessions, not the multi-day continuous run a production rollout would need to prove itself against, so every figure reported here describes one sitting rather than sustained operation. A single, broadly scoped IAM role covers both Lambda functions rather than each holding its own least-privilege policy -- a limit of the Learner Lab account itself, covered in Section II-F, not a choice this project would otherwise have made. And the automated integration suite still targets LocalStack rather than the live account, so everything this section claims about the real deployment rests on manual AWS CLI checks and browser inspection instead of an automated pass against production.

V. Conclusion

This project set out to answer a narrow question against a real slope-monitoring scenario: can a fog gateway watching two slopes, an SQS-fronted queue, two independently deployed Lambda functions, and a static S3 dashboard actually stand up as a running system on AWS, not just as a design on paper. By the time this account of it closes, the answer is yes, but only because every layer was pushed onto real infrastructure and checked there -- the fog gateway keeps no web framework in its path, the backend never shares one handler between ingestion and query serving, and each architectural choice along the way was weighed against a genuine alternative rather than asserted without one.

Five defects surfaced before this was true, and they split cleanly into two kinds. The DynamoDB pagination undercount and the missing SQS batching sailed through every local test and the full LocalStack integration run, because neither one changes behaviour at this project's small local data volume -- they only bite once a real table outgrows a single Scan page or a real flush cycle closes more than ten groups at once, conditions no local test ever creates. The remaining three -- the deploy-script credential risk, the credential-gating bug, and the missing CORS header -- were never about data volume at all. Two of them exist only because a real Lambda's execution environment and a real browser's security model behave in ways no local fake reproduces, and neither was found by writing another test; both were found by pointing the deployed code at the actual AWS account and the rendered page at an actual browser.

The hardest part of this project was not the application code itself. It was learning not to trust a green /api/health response as proof that a user's browser was seeing the same thing -- a lesson this project did not get to learn in the abstract, since its own health endpoint reported every field true while the dashboard in front of it rendered nothing at all. Two changes would matter most from here: moving the sensor and fog tier off one continuously running EC2 instance onto a scheduled, fully event-driven model, and, if this account's IAM restrictions were ever relaxed, splitting the shared LabRole into two least-privilege roles, one scoped to ska-processor and one to ska-dashboard-api.

VI. References

[1] F. Bonomi, R. Milito, J. Zhu, and S. Addepalli, “Fog computing and its role in the Internet of Things,” in Proc. 1st ACM Mobile Cloud Computing Workshop (MCC ’12), Helsinki, Finland, Aug. 2012, pp. 13–16.

[2] P. Mell and T. Grance, “The NIST Definition of Cloud Computing,” National Institute of Standards and Technology, Gaithersburg, MD, USA, Special Publication 800-145, Sep. 2011.

[3] A. van Herwijnen and J. Schweizer, “Monitoring avalanche activity using a seismic sensor,” Cold Regions Science and Technology, vol. 69, no. 2–3, pp. 165–176, 2011.

[4] S. Tilkov and S. Vinoski, “Node.js: Using JavaScript to build high-performance network programs,” IEEE Internet Computing, vol. 14, no. 6, pp. 80–83, Nov. 2010.

[5] G. DeCandia et al., “Dynamo: Amazon’s highly available key-value store,” in Proc. 21st ACM SIGOPS Symp. Operating Systems Principles (SOSP ’07), Stevenson, WA, USA, Oct. 2007, pp. 205–220.

[6] M. Shahrad et al., “Serverless in the wild: Characterizing and optimizing the serverless workload at a large cloud provider,” in Proc. 2020 USENIX Annu. Tech. Conf. (USENIX ATC ’20), Jul. 2020, pp. 205–218.

[7] J. Chen, J. Jiang, H. Duan, T. Wan, S. Chen, V. Paxson, and M. Yang, “We still don’t have secure cross-domain requests: An empirical study of CORS,” in Proc. 27th USENIX Security Symp. (USENIX Security ’18), Baltimore, MD, USA, Aug. 2018, pp. 1079–1093.